

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN - ỨNG DỤNG



BÀI TẬP CÁ NHÂN
HỌC PHẦN: HỌC SÂU (858048)

Giảng viên đảm nhiệm: TS. Đỗ Như Tài

Họ và tên sinh viên: Trần Quốc Hoàng

Mã số sinh viên: 3123580015

Lớp: DDU1231

Thành phố Hồ Chí Minh, ngày 16 tháng 09 năm 2026

BÀI 1: LÀM QUEN VỚI PYTORCH

1. Mục tiêu

- Sau khi hoàn thành bài thực hành, sinh viên có thể:
 - Tạo, kiểm tra và thao tác với Tensor trong PyTorch.
 - Thực hiện indexing, slicing, reshape và các phép toán ma trận trên Tensor.
 - Sử dụng các hàm thống kê cơ bản trên Tensor.
 - Hiểu cơ chế automatic differentiation (autograd) và vai trò của gradient.
 - Sử dụng optimizer để giải bài toán tối ưu đơn giản bằng Gradient Descent.
 - PyTorch cung cấp hai khả năng nền tảng: tính toán Tensor (tương tự NumPy, có thể tăng tốc bằng GPU) và
 - automatic differentiation để huấn luyện mô hình.
-

2. Bài tập thực hành

Bài 1. Tạo Tensor cơ bản

Tạo một Tensor số nguyên kích thước 2×3 và hiển thị kết quả.

```
a = torch.tensor(  
    [[1, 2, 3],  
     [4, 5, 6]],  
    dtype=torch.int32  
)  
print(a)
```

```
tensor([[1, 2, 3],  
        [4, 5, 6]], dtype=torch.int32)
```

Yêu cầu: Cho biết kích thước, số chiều và kiểu dữ liệu của Tensor a.

```
print("Kích thước:", a.shape)  
print("Số chiều:", a.ndim)  
print("Kiểu dữ liệu:", a.dtype)
```

```
Kích thước: torch.Size([2, 3])  
Số chiều: 2  
Kiểu dữ liệu: torch.int32
```

Nhận xét: Tensor a có kích thước là `torch.Size([2, 3])`, nghĩa là có 2 hàng và 3 cột. Vì vậy, Tensor a có 2 chiều và kiểu dữ liệu là `torch.int32`.

Bài 2. Tạo các Tensor thường dùng

Thực hành tạo Tensor ngẫu nhiên, Tensor toàn số 0, toàn số 1 và ma trận đơn vị.

```
a = torch.rand(3, 4)
b = torch.randn(3, 4)
c = torch.zeros(2, 3)
d = torch.ones(2, 3)
e = torch.eye(4)
```

```
print(a)
print(b)
print(c)
print(d)
print(e)
```

```
tensor([[0.4556, 0.6323, 0.3489, 0.4017],
        [0.0223, 0.1689, 0.2939, 0.5185],
        [0.6977, 0.8000, 0.1610, 0.2823]])
tensor([[ 0.5684, -0.5966,  0.1820, -0.8567],
        [ 1.1006, -1.0712,  0.1227, -0.5663],
        [ 0.3731, -0.8920, -1.5091,  0.3704]])
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])
```

Yêu cầu:

- Nhận xét sự khác nhau giữa `torch.rand()` và `torch.randn()`.
- Tạo thêm một Tensor 5×5 chứa toàn giá trị 10.

Nhận xét sự khác nhau giữa `torch.rand()` và `torch.randn()`:

- '`torch.rand()`': tạo các giá trị theo phân bố đều trong khoảng $[0, 1)$.
- '`torch.randn()`': tạo các giá trị theo phân bố chuẩn với kỳ vọng 0 và độ lệch chuẩn 1.

```
# Tạo Tensor 5 x 5 chứa toàn giá trị 10
messi = torch.full((5, 5), 10)
print("Tensor 5 x 5 full 10:")
print(messi)
```

Tensor 5 x 5 full 10:

```
tensor([[10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10],
        [10, 10, 10, 10, 10]])
```

Bài 3. Kiểm tra thông tin Tensor

Sử dụng `shape`, `size()`, `ndim` và `dtype` để kiểm tra các thuộc tính của Tensor.

```
a = torch.zeros(2, 3, 4)

print("Tensor:", a)
print("Shape:", a.shape)
print("Size:", a.size())
print("Number of dimensions:", a.ndim)
print("Data type:", a.dtype)
```

```
Tensor: tensor([[[0., 0., 0., 0.],
                 [0., 0., 0., 0.],
                 [0., 0., 0., 0.]],

                [[0., 0., 0., 0.],
                 [0., 0., 0., 0.],
                 [0., 0., 0., 0.]])
Shape: torch.Size([2, 3, 4])
Size: torch.Size([2, 3, 4])
Number of dimensions: 3
Data type: torch.float32
```

Yêu cầu: Giải thích ý nghĩa của `torch.Size([2, 3, 4])`.

- **Ý nghĩa của `torch.Size([2, 3, 4])`:** `torch.Size([2, 3, 4])` mô tả một Tensor 3 chiều với chiều thứ nhất có 2 phần tử, chiều thứ hai có 3 phần tử và chiều thứ ba có 4 phần tử. Tổng số phần tử là $2 \times 3 \times 4 = 24$.

Bài 4. Slicing và Indexing

Truy xuất một phần dữ liệu từ Tensor nhiều chiều.

```
a = torch.randn(3, 4, 5)

print("Original tensor:", a)
print("Tensor a[1]:", a[1])
print("Tensor a[1:, 2:4]:", a[1:, 2:4])
```

```
Original tensor: tensor([[-1.7396e-01, -6.7875e-01,  9.3826e-01,  4.8887e-01, -
6.7309e-01],
                          [ 8.7283e-01,  1.0554e+00,  1.7784e-01, -5.1807e-01, -3.0670e-01],
                          [-1.5810e+00,  1.7066e+00, -4.4622e-01,  7.4402e-01,  1.5210e+00],
                          [ 3.4105e+00,  5.9432e-01,  1.5419e+00,  5.0733e-01, -5.9103e-01]],

                        [[-5.6925e-01,  9.1997e-01,  1.1108e+00,  1.2899e+00, -1.4959e+00],
                          [-1.9384e-01,  4.4551e-01,  1.3253e+00, -1.6293e+00, -5.4974e-01],
                          [-4.7983e-01, -4.9968e-01, -1.1256e+00, -3.1700e-01, -1.0925e+00],
                          [-8.5194e-02, -9.3348e-02,  6.8705e-01, -8.3832e-01,  8.9182e-04]],

                        [[-7.5043e-01,  1.8541e-01,  6.2114e-01,  6.3818e-01,  7.3246e-02],
                          [ 1.1133e+00,  2.8227e-01,  4.3423e-01, -8.0249e-01, -1.2952e+00],
                          [-7.5018e-01, -1.3120e+00,  2.0642e-01, -3.3345e-01, -4.2883e-01],
```

```

[ 2.3292e-01,  7.9689e-01, -1.8484e-01, -3.7015e-01, -1.2103e+00]]])
Tensor a[1]: tensor([[ -5.6925e-01,  9.1997e-01,  1.1108e+00,  1.2899e+00, -
1.4959e+00],
[ -1.9384e-01,  4.4551e-01,  1.3253e+00, -1.6293e+00, -5.4974e-01],
[ -4.7983e-01, -4.9968e-01, -1.1256e+00, -3.1700e-01, -1.0925e+00],
[ -8.5194e-02, -9.3348e-02,  6.8705e-01, -8.3832e-01,  8.9182e-04]])
Tensor a[1:, 2:4]: tensor([[[ -4.7983e-01, -4.9968e-01, -1.1256e+00, -3.1700e-01,
-1.0925e+00],
[ -8.5194e-02, -9.3348e-02,  6.8705e-01, -8.3832e-01,  8.9182e-04]],
[[ -7.5018e-01, -1.3120e+00,  2.0642e-01, -3.3345e-01, -4.2883e-01],
[ 2.3292e-01,  7.9689e-01, -1.8484e-01, -3.7015e-01, -1.2103e+00]]])

```

Yêu cầu:

- Cho biết shape của `a[1]` và `a[1:, 2:4]`.
- Thử lấy hai phần tử cuối cùng trên chiều thứ ba.

```

print("Shape của a[1]:", a[1].shape)
print("Shape của a[1:, 2:4]:", a[1:, 2:4].shape)

# Dấu -2: lấy hai phần tử cuối cùng trên chiều thứ ba.
hai_phan_tu_cuoi = a[:, :, -2:]
print("Hai phần tử cuối trên chiều thứ ba:", hai_phan_tu_cuoi)
print("Shape:", hai_phan_tu_cuoi.shape)

```

```

Shape của a[1]: torch.Size([4, 5])
Shape của a[1:, 2:4]: torch.Size([2, 2, 5])
Hai phần tử cuối trên chiều thứ ba: tensor([[[ 4.8887e-01, -6.7309e-01],
[ -5.1807e-01, -3.0670e-01],
[ 7.4402e-01,  1.5210e+00],
[ 5.0733e-01, -5.9103e-01]],
[[ 1.2899e+00, -1.4959e+00],
[ -1.6293e+00, -5.4974e-01],
[ -3.1700e-01, -1.0925e+00],
[ -8.3832e-01,  8.9182e-04]],
[[ 6.3818e-01,  7.3246e-02],
[ -8.0249e-01, -1.2952e+00],
[ -3.3345e-01, -4.2883e-01],
[ -3.7015e-01, -1.2103e+00]])])
Shape: torch.Size([3, 4, 2])

```

Nhận xét shape của `a[1]` và `a[1:, 2:4]`:

- ‘`a[1]`’: chọn phần tử thứ hai trên chiều đầu tiên nên loại bỏ chiều đó và có shape (4, 5).
- ‘`a[1:, 2:4]`’: chọn 2 phần tử trên chiều đầu và 2 phần tử trên chiều thứ hai, nên shape là (2, 2, 5).

Bài 5. Reshape và Flatten

Thay đổi hình dạng Tensor mà không làm thay đổi số phần tử.

```

a = torch.randn(3, 4)

print("Original:", a)
print("Flatten:", a.ravel())
print("Reshape to 3 x 2 x 2:", a.reshape(3, 2, 2))

```

```
Original: tensor([[ 1.4565, -0.2849, -0.1334,  1.8929],
                  [ 3.1110, -0.4584, -0.3360, -1.5700],
                  [ 1.2315,  1.3946,  1.1711,  0.4335]])
Flatten: tensor([ 1.4565, -0.2849, -0.1334,  1.8929,  3.1110,
                  -0.4584, -0.3360, -1.5700,  1.2315,  1.3946,  1.1711,  0.4335])
Reshape to 3 x 2 x 2: tensor([[[ 1.4565, -0.2849],
                               [-0.1334,  1.8929]],

                              [[ 3.1110, -0.4584],
                               [-0.3360, -1.5700]],

                              [[ 1.2315,  1.3946],
                               [ 1.1711,  0.4335]]])
```

Yêu cầu:

- Kiểm tra số phần tử trước và sau khi reshape.
- Thử reshape Tensor thành kích thước 2×6 .

```
print("Số phần tử trước reshape:", a.numel())

tensor_da_reshape = a.reshape(2, 6)
print("Tensor sau reshape thành 2 x 6:", tensor_da_reshape)
print("Số phần tử sau reshape:", tensor_da_reshape.numel())
print("Flatten có shape:", a.flatten().shape)
```

Số phần tử trước reshape: 12

```
Tensor sau reshape thành 2 x 6: tensor([[ 1.4565, -0.2849, -0.1334,  1.8929,  3.1110, -0.4584],
                                         [-0.3360, -1.5700,  1.2315,  1.3946,  1.1711,  0.4335]])
```

Số phần tử sau reshape: 12

Flatten có shape: torch.Size([12])

Nhận xét: Reshape chỉ thay đổi hình dạng Tensor, không thay đổi số lượng phần tử. Tensor ban đầu có 12 phần tử và Tensor sau khi reshape thành 2×6 cũng có 12 phần tử.

Bài 6. Phép toán và nhân ma trận

Thực hiện các phép toán phần tử và phép nhân ma trận bằng toán tử @.

```
a = torch.randn(2, 3)
b = torch.randn(2, 3)

print("a:", a)
print("b:", b)
print("a + b:", a + b)
print("a / b:", a / b)
print("a^2:", a ** 2)
print("Matrix multiplication:", a @ b.T)
```

```

a: tensor([[ -1.7343, -1.3360,  0.8871],
          [ 0.7680,  0.0571,  0.2240]])
b: tensor([[ 0.5520, -0.5788,  0.0177],
          [ 0.1318,  1.0198, -0.4468]])
a + b: tensor([[ -1.1823, -1.9148,  0.9048],
              [ 0.8998,  1.0769, -0.2229]])
a / b: tensor([[ -3.1420,  2.3083, 50.0559],
              [ 5.8255,  0.0560, -0.5012]])
a^2: tensor([[ 3.0076,  1.7850,  0.7869],
            [ 0.5898,  0.0033,  0.0502]])
Matrix multiplication: tensor([[ -0.1682, -1.9875],
                              [ 0.3948,  0.0594]])

```

Yêu cầu:

- Phân biệt phép toán $a * b$ với $a @ b.T$.
- Cho biết shape của kết quả $a @ b.T$.

Phân biệt phép toán giữa $a * b$ và $a @ b.T$:

- ' $a * b$ ' là phép nhân từng phần tử. Nên kết quả vẫn giữ nguyên shape là (2, 3).
- ' $a @ b.T$ ' là tích vô hướng giữa các hàng, mỗi phần tử của kết quả là tích vô hướng giữa một hàng của a và một hàng của b . ((2, 3) @ (3, 2), nên kết quả có shape là (2, 2)).

```

print("Shape của a * b:", (a * b).shape)
print("Shape của a @ b.T:", (a @ b.T).shape)

```

```

Shape của a * b: torch.Size([2, 3])
Shape của a @ b.T: torch.Size([2, 2])

```

Bài 7. Các hàm thống kê trên Tensor

Tính trung bình, độ lệch chuẩn và tổng tích lũy theo từng cột.

```

a = torch.randn(3, 4)

print(a)
print("\nMean:")
print(torch.mean(a, dim=0))
print("Standard deviation:")
print(torch.std(a, dim=0))
print("Cumulative sum:")
print(torch.cumsum(a, dim=0))

tensor([[ 0.4520, -0.9759,  0.7112, -0.7582],
        [-0.6436, -0.6462, -0.1591, -1.7787],
        [ 0.8477,  0.2459, -0.1312, -0.1785]])

```

```

Mean:
tensor([ 0.2187, -0.4587,  0.1403, -0.9051])
Standard deviation:
tensor([0.7725, 0.6321, 0.4946, 0.8101])

```

```
Cumulative sum:
tensor([[ 0.4520, -0.9759,  0.7112, -0.7582],
        [-0.1916, -1.6221,  0.5521, -2.5369],
        [ 0.6561, -1.3761,  0.4210, -2.7154]])
```

Yêu cầu:

- Thay `dim=0` bằng `dim=1` và nhận xét sự khác nhau.

```
# dim=1 tính theo từng hàng
# dim=0 tính theo từng cột
print("Mean theo hàng (dim=1):", torch.mean(a, dim=1))
print("Standard deviation theo hàng (dim=1):", torch.std(a,
dim=1))
print("Cumulative sum theo hàng (dim=1):", torch.cumsum(a,
dim=1))
```

```
Mean theo hàng (dim=1): tensor([-0.1427, -0.8069,  0.1960])
Standard deviation theo hàng (dim=1): tensor([0.8477, 0.6871, 0.4741])
Cumulative sum theo hàng (dim=1): tensor([[ 0.4520, -0.5239,  0.1873,
-0.5709],
        [-0.6436, -1.2897, -1.4488, -3.2275],
        [ 0.8477,  1.0936,  0.9624,  0.7839]])
```

Sự khác nhau giữa `dim=0` và `dim=1`:

- `dim=0`: phép tính được thực hiện theo từng cột nên kết quả của ‘mean’ và ‘std’ có 4 phần tử; ‘cumsum’ cộng dồn theo chiều từ trên xuống dưới.
- `dim=1`: phép tính được thực hiện theo từng hàng nên ‘mean’ và ‘std’ có 3 phần tử; ‘cumsum’ cộng dồn từ trái sang phải trong mỗi hàng.

Bài 8. Autograd cơ bản

Tạo biến có `requires_grad=True` và dùng `backward()` để tính đạo hàm tự động.

```
x = torch.tensor(
    3.6,
    requires_grad=True
)

y = x * x
y.backward()

print("x =", x)
print("y =", y)
print("x.grad =", x.grad)
```

```
x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)
```

Yêu cầu:

- Tính thủ công đạo hàm của $y = x^2$ tại $x = 3.6$ và so sánh với `x.grad`.
- Đổi hàm thành $y = 3x^2 + 2x + 1$ và kiểm tra gradient.

```
# Với  $y = x^2$ 
print("Đạo hàm thủ công của  $y = x^2$  tại  $x = 3.6$ :", 2 * x.item())
print("Gradient do autograd tính:", x.grad.item())

# Tạo một Tensor mới để gradient của hàm mới không cộng dồn với
# gradient cũ.
x_new = torch.tensor(3.6, requires_grad=True)
y_new = 3 * x_new**2 + 2 * x_new + 1
y_new.backward()

# Với  $y = 3x^2 + 2x + 1$ 
print("Đạo hàm thủ công của hàm mới:", 6 * x_new.item() + 2)
print("Gradient của hàm mới:", x_new.grad.item())
```

Đạo hàm thủ công của $y = x^2$ tại $x = 3.6$: 7.199999809265137

Gradient do autograd tính: 7.199999809265137

Đạo hàm thủ công của hàm mới: 23.59999942779541

Gradient của hàm mới: 23.599998474121094

Nhận xét:

- Với $y=x^2$, ta có $dy/dx = 2x$, nên tại $x=3.6$ ta sẽ có đạo hàm bằng 7.2.

Kết quả của 'x.grad' trùng với phép tính thủ công.

- Với $y=3x^2 + 2x + 1$, đạo hàm là $6x+2$ và tại $x=3.6$ nên ta sẽ có đạo hàm bằng 23.6, cũng trùng với kết quả của autograd.

Bài 9. Gradient Descent tìm hệ số đa thức

Sinh dữ liệu từ đa thức $y = x^2 + 2x + 3$, sau đó dùng NAdam và autograd để tìm lại ba hệ số.

```
polynomial = np.poly1d([1, 2, 3])
N = 20

X = np.random.randn(N, 1) * 5
Y = polynomial(X)

XX = np.hstack([
    X * X,
    X,
    np.ones_like(X)
])

w = torch.randn(3, 1, requires_grad=True)
x = torch.tensor(XX, dtype=torch.float32)
y = torch.tensor(Y, dtype=torch.float32)

optimizer = torch.optim.NAdam([w], lr=0.01)
print("Initial coefficients:")
print(w)

for epoch in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()
print("Learned coefficients:")
print(w)
```

```
Initial coefficients:
tensor([[ -0.5959],
        [ 0.2739],
        [ 0.5679]], requires_grad=True)
Learned coefficients:
tensor([[1.0108],
        [1.9721],
        [2.3446]], requires_grad=True)
```

Yêu cầu:

- So sánh ba hệ số học được với [1, 2, 3].
- Giải thích vai trò của `optimizer.zero_grad()`, `mse.backward()` và `optimizer.step()`.

```
# Các hệ số kỳ vọng là [1, 2, 3], tương ứng với x^2, x và hằng số.
he_so_hoc_duoc = w.detach().flatten()
he_so_mong_doi = torch.tensor([1.0, 2.0, 3.0])
print("Hệ số kỳ vọng:", he_so_mong_doi)
print("Hệ số học được:", he_so_hoc_duoc)
print("Sai số tuyệt đối từng hệ số:", torch.abs(he_so_hoc_duoc - he_so_mong_doi))
```

```
Hệ số kỳ vọng: tensor([1., 2., 3.])
Hệ số học được: tensor([1.0108, 1.9721, 2.3446])
Sai số tuyệt đối từng hệ số: tensor([0.0108, 0.0279, 0.6554])
```

Nhận xét và giải thích: Các hệ số học được gần với [1, 2, 3], tương ứng với đa thức $x^2 + 2x + 3$; sai khác rất nhỏ do tính toán số thực và quá trình tối ưu. Trong mỗi vòng lặp:

- ‘`optimizer.zero_grad()`’: xóa gradient của vòng lặp trước để gradient không bị cộng dồn.
- ‘`mse.backward()`’: dùng autograd để tính gradient của hàm mất mát theo các hệ số.
- ‘`optimizer.step()`’: cập nhật các hệ số theo gradient và learning rate của NAdam nhằm giảm MSE.

Bài 10. Giải hệ phương trình bằng Autograd

Tối thiểu hóa tổng sai số bình phương để tìm A, B, C, D thỏa bốn phương trình.

```
# Cố định seed để nghiệm số có thể tái lập giữa các lần chạy.
random.seed(0)
A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)

EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)

for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2

    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4

    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print("A =", A)
print("B =", B)
print("C =", C)
print("D =", D)
```

```
A = tensor(4.5494, requires_grad=True)
B = tensor(4.4506, requires_grad=True)
C = tensor(3.4506, requires_grad=True)
D = tensor(2.4506, requires_grad=True)
```

Yêu cầu:

- Kiểm tra nghiệm thu được với bốn phương trình: $A+B=9$, $C-D=1$, $A+C=8$, $B-D=2$.
- Nhận xét vì sao bài toán có thể có nhiều nghiệm.

```
# Kiểm tra trực tiếp bốn vế trái của hệ phương trình sau khi tối ưu.
sai_so_he = torch.stack([
    A + B - 9,
    C - D - 1,
    A + C - 8,
    B - D - 2,
])
print("Sai số của bốn phương trình:", sai_so_he.detach())
print("Sai số tuyệt đối lớn nhất:", sai_so_he.abs().max().item())
print("A + B =", (A + B).item(), "| cần bằng 9")
print("C - D =", (C - D).item(), "| cần bằng 1")
print("A + C =", (A + C).item(), "| cần bằng 8")
print("B - D =", (B - D).item(), "| cần bằng 2")
```

```
Sai số của bốn phương trình: tensor([-4.7684e-05,  1.0490e-05, -
3.0994e-05, -6.6757e-06])
```

```
Sai số tuyệt đối lớn nhất: 4.76837158203125e-05
```

```
A + B = 8.99995231628418 | cần bằng 9
C - D = 1.0000104904174805 | cần bằng 1
A + C = 7.999969005584717 | cần bằng 8
B - D = 1.9999933242797852 | cần bằng 2
```

Nhận xét:

- Các giá trị tìm được kiểm tra đúng bốn phương trình với sai số rất nhỏ do tối ưu số. Hệ không có nghiệm duy nhất vì chỉ có ba phương trình độc lập.

- Đặt $A=t$ thì: $B=9-t$, $C=8-t$, $D=7-t$.

=> Vì t có thể nhận nhiều giá trị, nên hệ có vô số nghiệm.

3. Bài tập áp dụng

Bài 11. Viết chương trình PyTorch để thực hiện các yêu cầu

- Tạo Tensor X có 24 phần tử và reshape lần lượt thành 4×6 , 2×12 và $2 \times 3 \times 4$.
- Tạo hai Tensor A, B kích thước 3×4 ; tính $A+B$, $A*B$, A/B , trung bình theo cột và độ lệch chuẩn theo cột.
- Cho $y = 2x^2 + 5x + 3$. Dùng autograd tính dy/dx tại $x = 2$ và kiểm tra bằng phép tính thủ công.
- Thay đa thức trong Bài 9 bằng $y = 2x^2 - 3x + 5$; huấn luyện và cho biết các hệ số tìm được.

```
# Bài 11.1: Reshape giữ nguyên 24 phần tử.
X = torch.arange(24)
print("X ban đầu:", X)
for shape in [(4, 6), (2, 12), (2, 3, 4)]:
    reshaped_X = X.reshape(shape)
    print(f"X reshape {shape}: shape={reshaped_X.shape},
          numel={reshaped_X.numel()}")
```

```
X ban đầu: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11,
                  12, 13, 14, 15, 16, 17,
                  18, 19, 20, 21, 22, 23])
X reshape (4, 6): shape=torch.Size([4, 6]), numel=24
X reshape (2, 12): shape=torch.Size([2, 12]), numel=24
X reshape (2, 3, 4): shape=torch.Size([2, 3, 4]), numel=24
```

```
# Bài 11.2: A * B là phép nhân từng phần tử; mean/std theo dim=0
là theo cột.
A = torch.randn(3, 4)
B = torch.randn(3, 4)
print("A + B:\n", A + B)
print("A * B:\n", A * B)
print("A / B:\n", A / B)
print("Trung bình theo cột:\n", torch.mean(A, dim=0))
print("Độ lệch chuẩn theo cột:\n", torch.std(A, dim=0))
```

```
A + B:
tensor([[ 0.6911, -1.6508, -1.0255, -0.4886],
        [-0.6711, -0.3489,  1.0532,  0.3432],
        [ 0.9569, -0.6648,  2.3644, -2.3616]])
```

```

A * B:
  tensor([[ 0.0697,  0.6142, -0.5218, -0.3598],
          [-1.2647, -0.2664, -0.5875, -0.5607],
          [ 0.1411, -0.1644,  1.3909,  1.3823]])
A / B:
  tensor([[ 4.6326,  1.9150, -3.7484, -0.4522],
          [-0.5553, -1.9419, -0.2769, -0.6348],
          [ 0.2349, -4.4649,  0.8709,  0.8301]])
Trung bình theo cột:
  tensor([ 0.5295, -0.8868, -0.2338, -0.4215])
Độ lệch chuẩn theo cột:
  tensor([0.3297, 0.1845, 1.2582, 0.7527])

```

```

# Bài 11.3: Autograd cho  $y = 2x^2 + 5x + 3$ , ta có  $dy/dx = 4x + 5$ .
# Tại  $x=2$ , đạo hàm sẽ bằng 13.
x = torch.tensor(2.0, requires_grad=True)
y = 2 * x**2 + 5 * x + 3
y.backward()
thucong = 4 * x.item() + 5
print("dy/dx bằng autograd:", x.grad.item())
print("dy/dx thủ công =  $4x + 5$ :", thucong)

```

```

dy/dx bằng autograd: 13.0
dy/dx thủ công =  $4x + 5$ : 13.0

```

```

# Bài 11.4: Dùng mô hình tuyến tính theo các đặc trưng [ $x^2$ ,  $x$ , 1]
# Học ba hệ số của đa thức  $y = 2x^2 - 3x + 5$ .
torch.manual_seed(0)
X_poly = torch.linspace(-5, 5, 40).reshape(-1, 1)
Y_poly = 2 * X_poly**2 - 3 * X_poly + 5
features = torch.cat([X_poly**2, X_poly,
                     torch.ones_like(X_poly)], dim=1)
coefficients = torch.randn(3, 1, requires_grad=True)
poly_optimizer = torch.optim.NAdam([coefficients], lr=0.01)

# Tăng số bước để NAdam hội tụ đủ ở cả hệ số  $x^2$  và hệ số tự do.
for _ in range(5000):
    poly_optimizer.zero_grad()
    predictions = features @ coefficients
    loss = torch.mean((predictions - Y_poly) ** 2)
    loss.backward()
    poly_optimizer.step()

print("Hệ số học được [ $x^2$ ,  $x$ , 1]:",
      coefficients.detach().flatten())
print("Hệ số mong đợi [2, -3, 5]:", torch.tensor([2.0, -3.0, 5.0]))
print("MSE cuối:", loss.item())

```

```

Hệ số học được [ $x^2$ ,  $x$ , 1]: tensor([ 2.0000, -3.0000,  5.0000])
Hệ số mong đợi [2, -3, 5]: tensor([ 2., -3.,  5.])
MSE cuối: 2.880099769786426e-10

```

Nhận xét bài 11.4: Mô hình dùng ba đặc trưng là $[x^2, x, 1]$, nên ba hệ số học được lần lượt gần với $[2, -3, 5]$. MSE gần 0 cho thấy đa thức đã được học đúng.